Computer Science Revision Notes

C. Irving

Contents

1 Programming **1**
- 1.1 Variables . . . 1
- 1.2 Input/Output . . . 3
- 1.3 Comments . . . 4
- 1.4 Selection . . . 5
- 1.5 Iteration . . . 6
- 1.6 Arrays . . . 7
- 1.7 Records . . . 8

2 Data Structures **9**

3 Algorithms **11**

4 Computation **13**

5 Data Representation **15**
- 5.1 Number Systems . . . 15
  - 5.1.1 Casual Definitions . . . 15
  - 5.1.2 Formal Definitions . . . 16
- 5.2 Binary Numbers . . . 18
  - 5.2.1 Binary to Denary Conversion . . . 18
  - 5.2.2 Denary to Binary Conversion . . . 18
  - 5.2.3 Special Properties of Binary Numbers . . . 19
  - 5.2.4 Overflow . . . 19
  - 5.2.5 Fixed-Point Numbers . . . 19
- 5.3 Binary Arithmetic . . . 20
  - 5.3.1 Bit Shifts . . . 20
  - 5.3.2 Binary Addition . . . 20
- 5.4 Sound . . . 21
- 5.5 Images . . . 22
- 5.6 Compression . . . 23
- 5.7 Encryption . . . 24

6 Computer Systems **25**
6.1 Hardware and Software . . . 25
6.1.1 Application Software . . . 25
6.1.2 System Software . . . 25
6.1.3 Translators . . . 27
6.2 Operating Systems . . . 29
6.3 Programming Language Classification . . . 30
6.3.1 Machine Code . . . 30
6.3.2 Assembly . . . 30
6.3.3 High-Level Languages . . . 31
6.3.4 Interpreted Languages . . . 32
6.3.5 Compiled Languages . . . 32
6.4 Boolean Algebra . . . 33
6.4.1 Truth Tables . . . 33
6.4.2 Logic Gates . . . 33
6.4.3 De Morgan's Laws . . . 35
6.4.4 Other Laws of Boolean Algebra . . . 36
6.4.5 Order of Operations . . . 37

7 Organisation and Architecture **39**

8 Consequences of Use **41**

9 Communication and Networking **43**

10 Databases **45**

11 Big Data **47**

12 Functional Programming **49**

13 Problem Solving **51**

Chapter 1

Programming

1.1 Variables

A variable is a named location in memory that can store a value. Variables have data types, which determines what type of data can be stored in them, e.g. strings or integers. Variables can also be set with a value, which tells the program what to store in the variable. Giving a variable an initial value before it is used is called initialisation.

Variables in VB.Net are declared using
`Dim <variable name> As <datatype>`:

```
' Declare a variable caled 'foo' with type String
Dim foo As String

' Declare a variable called 'bar' with type String and
' initialise it with a value of "Hello World"
Dim bar As String = "Hello World"

' Declare a variable called 'baz' with type Integer and
' initialise it with a value of 0
Dim baz As Integer = 0
```

In C++, they are declared using `<datatype> <variable name>;`:

```
// Declare a variable called 'foo' with type std::string
std::string foo;

// Declare a variable called 'bar' with type std::string and
// initialise it with a value of "Hello World"
std::string bar = "Hello World";
```

```
// Declare a variable called 'baz' with type int and
// initialise it with a value of 0
int baz = 0;
```

After declaring a variable, its value can be changed. In VB.Net, this is done using `<variable name> = <new value>`, and in C++ this is done using `<variable name> = <new value>;`.

Calculations can also be done using variables, for example, this is how to add two variable in VB.Net:

```
Dim a As Integer = 2
Dim b As Integer = 5
Dim sum As Integer = a + b ' Add a and b and store the sum
```

And this is it in C++:

```
int a = 2;
int b = 5;
int sum = a + b; // Add a and b and store the sum
```

Other operations such as addition, subtraction, multiplication and division can be done using `+`, `-`, `*` and `/`. As well as these, there is also the modulo and integer divison operations.

Modulo (`Mod` in VB.Net, `%` in C++) takes the first number, divides it by the second number, then takes the remainder. For example, $7 \mod 3 = 1$, because $7 \div 3 = 2$ remainder 1.

Integer divison (\ in VB.Net) takes the first number and divides it by the second number, but rounds down so that it only takes the integer part. It does not have a specific symbol in C++, so it done by `Math.floor(a / b)`, using the `Math` library for the floor function.

1.2 Input/Output

Input and output are how the program can talk to the user. Input takes in data that the user enters, and output gives data back to the user.

Taking an input in VB.Net is done using `foo = Console.ReadLine()`, and in C++, it is done using `std::cin >> foo;`.

Outputting text in VB.Net is done using `Console.WriteLine(bar)`, and in C++, it is done using `std::cout << bar;`.

Using input and output together can take data in, transform it somehow, and then output it. This an example in VB.Net:

```
' Prompt the user to enter their name
Console.WriteLine("Enter your name")

' Take the user's input and store it to a variable called 'name'
Dim name As String = Console.ReadLine()

' Output Hello and the name that they input.
Console.WriteLine("Hello " & name)
```

This is the same example in C++:

```
// Prompt user to enter their name
// \n leaves a new line after the input
std::cout << "Enter your name" << \n;

// Take the user's input and store it to a variable called 'name'
std::string name;
std::cin >> name;

// Output Hello and the name that they input.
std::cout << "Hello " + name;
```

The two examples have to join strings together, which is called concatenation. In VB.Net, this is done using `&`, and in C++, it is done using `+`. This creates a new string made up of the other two. For example, concatenating `"Hello "` and `"World"` would give the string `"Hello World"`. Note that the `"Hello "` string has a space on the end, meaning that there is a space between the two words in the concatenated string.

1.3 Comments

1.4 Selection

1.5 Iteration

1.6 Arrays

1.7 Records

Chapter 2

Data Structures

Chapter 3

Algorithms

Chapter 4

Computation

Chapter 5

Data Representation

5.1 Number Systems

5.1.1 Casual Definitions

Numbers can be categorised into many different groups, starting with the natural numbers. The natural numbers are the numbers used for counting, 0, 1, 2,.... The set of nautral numbers is written:

$$\mathbb{N} = \{0, 1, 2, \dots\} \tag{5.1}$$

The next set is the set of integers ($\mathbb{Z}$). This is the set of all real numbers without a decimal part, including all natural numbers (0, 1, 2,...), and all of the negatives of the natural numbers (-1, -2,...).

$$\mathbb{Z} = \{0, 1, -1, 2, -2, \dots\} \tag{5.2}$$

After the integers, the rational numbers can be defined as the set of all ratios $\frac{a}{b}$, where a and b are integers, and $b \neq 0$. The set of rational numbers is written $\mathbb{Q}$:

$$\mathbb{Q} = \left\{ \frac{a}{b} : a, b \in \mathbb{Z}, b \neq 0 \right\} \tag{5.3}$$

The set of real numbers ($\mathbb{R}$) is the set of all numbers in the interval $(-\infty, \infty)$, i.e. the whole number line.

Each of these sets are proper subsets of each other, with $\mathbb{N}$ being the smallest, and $\mathbb{R}$ being the largest. A subset is defined such that is A is a subset of B ($A \subseteq B$), then B contains every element in A, and A only contains elements in B. Proper subsets are subsets such that $A \subseteq B$ and $A \neq B$, and is written $A \subsetneq B$.

$$\mathbb{N} \subsetneq \mathbb{Z} \subsetneq \mathbb{Q} \subsetneq \mathbb{R} \tag{5.4}$$

Because of how subsets are defined, if $A \subset B \subset C$, then $A \subset C$ ($\subset$ referring to either $\subseteq$ or $\subsetneq$).

As well as these, there are some other notable sets:

The set of all positive integers is written $\mathbb{Z}^+$, which is the same as the set of natural numbers excluding 0[1]:

$$\mathbb{Z}^+ = \mathbb{N} \setminus \{0\} \tag{5.5}$$

The empty set is written $\emptyset$:

$$\emptyset = \{\} \tag{5.6}$$

Since the empty set contains no elements, any set contains every element of $\emptyset$, and $\emptyset$ does not contain any elements not in any given set, so $\emptyset \subseteq A$ for any set A.

The irrational numbers is the set of all real numbers that aren't rational:

$$\text{The set of irrational numbers} = \mathbb{R} \setminus \mathbb{Q} \tag{5.7}$$

There is no widely-accepted symbol for the set of irrational numbers, so $\mathbb{R} \setminus \mathbb{Q}$ is often used.

Ordinal numbers are used to order things, these are the numbers 1st, 2nd, 3rd. The ordinal numbers are similar to the natural numbers, but instead of being used to count objects, they are used to order objects.

5.1.2 Formal Definitions

These definitions are **not** required for A-Level.

The Natural Numbers

We will define 0 as the empty set, $0 = \{\}$. Now, we will define the successor of any natural number n as $S(n) = \{n, \{n\}\}$. This means that the successor of 0 is $S(0) = \{0, \{0\}\} = \{\{\}, \{\{\}\}\}$. In casual terms, $S(n) = n + 1$, so $S(0)$ is the definition of 1 in the natural numbers. Now, the set of natural numbers is defined as the set containing 0 such that for all elements $n \in \mathbb{N}$, $S(n) \in \mathbb{N}$. This means that $S(0) \in \mathbb{N}$, which means that $S(S(0)) = S(1) \in \mathbb{N}$, etc.

The Integers

Every integer can be written as a pair of natural numbers, (n_1, n_2) such that $\mathbb{Z} \ni x = n_1 - n_2$.

Proof. For $x \in \mathbb{N}$, x can be written as $x - 0$, so $x = (x, 0)$. For the negative numbers, the opposite can be used; for some integer $x < 0$, x must be a natural number, since $-x$ would be a positive integer if x is a negative integer, and all positive integers are also natural numbers. Thus, x can be written as $0 - (-x)$, so $x = (0, -x)$. Because all natural numbers and all negative integers can be written in this form, that means that all integers can be written in this form ($\mathbb{N} \cup \mathbb{Z}^- = \mathbb{Z}$). □

[1] The symbol $\setminus$ means excluding, i.e. $A \setminus B$ is the set containing all elements of A that are not in B or, formally: $A \setminus B = \{x \in A : x \notin B\}$.

However, this definition means that there are duplicate integers, (e.g. $2-1 = 3-2 = 1$), so we will define an equivalence relation between the integers.

Let $\sim$ be an equivalence relation such that $(a,b) \sim (c,d)$ iff[2] $a-b = c-d$. This means that $(a,b) \sim (c,d)$ iff $a+d = b+c$.

Addition in the natural numbers can be defined as follows:

$x+0 = x$ for any natural number x. For any $x, S(y) \in \mathbb{N}$, $x+S(y) = S(x+y)$.

From here, the integers will fall into equivalence classes (sets of integers such that each integer in a set is equal to all other integers in the set). The proof that $\sim$ is an equivalence relation is as follows:

Proof. By the definition of an equivalence relation: a relation $R \subseteq A^2$ is an equivalence relation iff $R(x,x) \,\forall\, x \in A$ and $R(x,y) \iff R(y,x) \,\forall\, x,y \in A$ and $R(x,y) \wedge R(y,z) \implies R(x,z) \,\forall\, x,y,z \in A$ (where $R(x,y)$ means $(x,y) \in A^2$).

Therefore if $\sim$ is an equivalence relation, for all $x,y,z \in \mathbb{Z}$: $x \sim x$, $x \sim y \iff y \sim x$ and $(x \sim y) \wedge (y \sim z) \implies (x \sim z)$.

- Criterion 1, $x \sim x$: let $x = (a,b)$, thus $x \sim x$ must be true, since $x \sim x \iff a+b = a+b$, which is clearly true.
- Criterion 2, $x \sim y \iff y \sim x$: let $x = (a,b)$ and $y = (c,d)$, thus $x \sim y$ means that $a+d = b+c$, and $y \sim x$ means that $d+a = c+b$. Since addition is commutative, $a+d = d+a = b+c = c+b$.
- Criterion 3, $(x \sim y) \wedge (y \sim z) \implies (x \sim z)$: let $x = (a,b)$, $y = (c,d)$ and $z = (e,f)$. $x \sim y$ can be rewritten as $a+d = b+c$, $y \sim z$ can be rewritten as $c+f = d+e$ and $x \sim z$ can be rewritten as $a+f = b+e$. Thus, if $a+d = b+c$ and $c+f = d+e$, then the two equations can be combined to make $(a+d)+(c+f) = (b+c)+(d+e)$. Removing the c and d term from each side then gives $a+f = b+e \implies x \sim z$.

Thus, all criteria hold, so $\sim$ is an equivalence relation. □

From this equivalence relation, the set of integers can be defined as the set of equivalence classes under $\sim$.

The Rationals

The Reals

[2] 'iff' means if and only if.

5.2 Binary Numbers

Binary numbers are numbers written in base-2, in contrast to the normal denary form (base-10). This means that binary has two digits, 0 and 1. Place value still works like denary, but with powers of 2 instead of powers of 10. This means that the rightmost digit has a place value of 2^0, the next one left has a place value of 2^1, the next one has a place value of 2^2, etc. Binary digits are also called bits.

5.2.1 Binary to Denary Conversion

Working out the value of a binary number means adding together all of the place with 1 digits (nothing has to be multiplied by the digit, since the only digits are 0 and 1, so anything with a 0 is just ignored, and anything with a 1 is added). For example, the denary value of the number 01011010_2 (the $_2$ means base-2) can be calculated as follows:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	1	0	1	0

Add each column with a 1 in: $2^6 + 2^4 + 2^3 + 2^1 = 64_{10} + 16_{10} + 8_{10} + 2_{10} = 90_{10}$ ($_{10}$ means base-10).

So $01011010_2 = 90_{10}$.

5.2.2 Denary to Binary Conversion

To convert back, find the greatest power of two smaller than the number you're converting. Using 90_{10} as an example, $2^7 = 128$ is greater then 90, but $2^6 = 64$ is smaller, so 2^6 is the greatest power of 2 less than 90. Now, subtract 2^6 from 90, and record a 1 in the 2^6 place of the binary number. Repeat with this process, until the number remaining minus the greatest power of two less than it = 0.

$$90 - 2^6 = 26$$
$$26 - 2^4 = 10$$
$$10 - 2^3 = 2$$
$$2 - 2^1 = 0$$

So the columns with 1s are the 2^6, 2^4, 2^3 and 2^1 columns, and the rest are 0s:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	1	0	1	0

So $90_{10} = 01011010_2$.

5.2.3 Special Properties of Binary Numbers

The largest number that can be stored in an n-digit binary number is found by setting all of the digits to 1. For example, the largest value that can be stored in 8 bits is 11111111_2, which is $2^7 + 2^6 + \cdots + 2^0 = 255$. This will always be 1 less than the place value of the $n + 1$th digit, since adding 1 to the number would force every bit to carry, adding a new bit on the left. Therefore, the largest value that can be stored in n bits is $2^n - 1$.

The furthest left and furthest right bits are called the most and least significant bits (MSB/LSB) respectively. The MSB (furthest left) has the greatest place value in the number, and it determines the greatest number that can be represented. The LSB (furthest right) has the least place value in the number ($2^0 = 1$), and this determines the parity of the number (whether it is odd or even). Since all places above the LSB are even (since they are all powers of 2), only the LSB can make the number odd, thus if the LSB is 1, the number is odd, and if it is 0, the number is even.

5.2.4 Overflow

5.2.5 Fixed-Point Numbers

5.3 Binary Arithmetic

Doing arithmetic with binary numbers is very similar to doing arithmetic with denary numbers, just with place values that are powers of 2, rather than powers of 10.

5.3.1 Bit Shifts

A bit shift is where all of the bits in a binary number are moved left or right, with zeros placed appropriately to preserve place value.

Left Shifts

A left shift moves all of the bits left by 1 bit, adding a zero to the furthest right bit (which would otherwise be left empty). Moving all of the bits left by 1 multiplies them all by 2, meaning that the overall effect is multiplying the number by 2. This is equivalent to mulitplication by 10 in denary, where all of the digits are moved left, and a zero is added at the end.

As an example of a left bit shift, this is $90_{10} = 01011010_2$ shifted left by 1 bit:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	1	1	0	1	0	0

Converting that back to denary gives $10110100_2 = 180_{10}$, meaning that the original 90 has been multiplied by 2.

Right Shifts

Similar to how a left shift multiplies a binary number by two, a right shift divides it by 2. Using integers, this is integer division, i.e. taking $3_{10} = 11_2$ and shifting it right by 1 gives $1_2 = 1_{10}$, not 1.5_{10}. However, if the number is also using decimal bits, the shift will move the LSB to the $2^{-1} = \frac{1}{2}$ bit.

As an example of a left bit shift, this is $90_{10} = 01011010_2$ shifted right by 1 bit:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	1	0	1	1	0	1

Converting that back to denary gives $10110100_2 = 45_{10}$, meaning that the original 90 has been divided by 2.

5.3.2 Binary Addition

5.4 Sound

5.5 Images

5.6 Compression

5.7 Encryption

Chapter 6

Computer Systems

6.1 Hardware and Software

Hardware is the electronic and electromechanical components of a computer, for example the CPU, the RAM or a monitor Software is code that runs on the hardware, for example apps or the operating system. Software does not have a phyisical presence.

6.1.1 Application Software

6.1.2 System Software

System software is software that runs to support application software.

Operating Systems

An operating system is a middle-man between the user/software and the hardware. Its tasks include:

Processor Scheduling Processor scheduling is how the computer decides what processes should be done and in what order. It does this by assigning priorities to each of the processes, and then uses an algorithm to decide which to do first.

The first-come first-serve algorithm (FIFO) immediately executes the first process that arrives, and all other processes join a first-in first-out queue structure to be executed in order. This can mean that high-priority processes or processes with short deadlines can be ignored for a long time if there are a lot of other processes already in the queue.

The earliest deadline first algorithm (EDF) works by finding the processes with the soonest to expire deadlines and executes them first.

The round robin algorithm allocates a unit of time to each process, and then cycles to the next one whether or not the first has finished, and then it will be

cycled back to later. These units of time are called time slices. A time slice, AKA a quantum, is a short unit of time where processes can execute, allowing for multitasking.

Fixed-priority preemptive scheduling is the algorithm used by Windows 11, and involves ranking each process based on priority. Factors such as deadlines can affect the priority of a process. The processes are then executed in order of priority, with any new high-priority processes jumping in front of the other, lower-priority ones.

Memory Management Memory management is a set of techniques used to keep a computer's memory organised and working as quickly as possible. This involves keeping track of what memory is in use and what memory is free.

Paging uses a lookup table called a page table to map logical addresses (the number assigned to the memory location) to the physical location (the physical place the data is stored), meaning that it can be stored anywhere, and is not stored in continguous pieces.

Segmentation stores data in continguous blocks based on their relations to each other, meaning that a program would be stored as one block of data in memory. A segment could store a program module, for example.

Virtual memory is where parts of the secondary storage is used as RAM. This memory will be slower, since secondary storage is not as fast as RAM, however computers generally have more secondary storage than RAM, since it is much cheaper. As RAM prices go up, virtual memory is becoming more useful for cheaper computers that have less RAM.

Disk thrashing is a performance drop caused by the computer swapping lots of data between secondary storage and RAM, which causes the CPU to slow down. This is in turn caused by bad memory management, which should aim to minimise disk thrashing by copying larger chunks of data from secondary storage to RAM, with the hope that the nearby data will also be useful. This can also be caused by not having enough RAM, which means that not enough data can be stored at once.

Memory management means that the computer cannot overwrite memory that is in-use, which would cause programs to have to fetch data back from secondary storage. It also allows multiple programs to occupy the same memory, which can be allocated properly to avoid data being overwritten. It also means that programmers don't have to worry about where their programs and data are stored, since it is all handled by the OS.

Backing Store Management a

IO Management a

6.1.3 Translators

Translators are programs the convert source code (written by the developer) from high-level languages into object code (the executable file containing machine code, which is executed to run the program). Translators generally fall into three categories:

- **Compilers** - a compiler translates all of the source code into an object code file before the program is run. This means that the object code can be distributed alone, stopping people from stealing the source code. Compiled code can be used across multiple different processors as long as there is a compiler for that processor family. This means that one source code file can be run on many different computers. Each time the code is changed, it requires recompiling, which can be a slow process for large programs, however this means that running it does not take any longer than just executing machine code, because the object code file already exists.

  A compiler goes through multiple phases: lexical analysis, syntactic analysis and semantic analysis of the code. This involves creating intermediate code, which is then optimised, before generating the machine code. If there are any errors in the syntax, it will not complete the compilation, and no machine code will be generated. The compiler will detect any syntax errors, which can be reported back to the developer.

  A compiler creates relocatable code, which means that the memory used in the program can be moved elsewhere in the computer. This also means that it can be run on different computers, using different memory addresses. This is different to assembly, which specifies specific addresses which can't be moved. It uses an offset (the point where the code starts) to determine where each other line of code is, no matter where the code is located.

- **Interpreters** - an interpreter goes through each line of the code as the code runs. Interpreters do not generate any machine code; they instead use their own subroutines to run the processes as they are read. The first part of the interpreting process is scanning for syntax errors, meaning that it will not waste time trying to run incorrect code. This means that the developer can see where errors are. Interpreters are useful because they can be used to test small pieces of code without having to compile a larger program. They are also good for teaching, using interactive mode. Interactive mode runs the code in a separate window, allowing you to see what it is doing at each step.

- **Assemblers** - assemblers convert each line of assembly into a set of obeject code bits. Assembly has a one-to-one correspondance with machine code, but each processor has its own set of instructions, meaning that code cannot be easily moved from one machine to another. This means different code will have to be written for each processor.

> Using assembly means that the code can't be moved in memory, since each memory location is hardcoded, meaning that if the code is moved, the goto statements will reference the wrong memory addresses.

Both compilers and interpreters are available for most high-level languages.

Bytecode

Bytecode is an intermediate form of source code. Bytecode is most famously used by Java/the JVM (Java Viryual Machine), where Java compiles to bytecode, which is then compiled into machine code by the JVM. Bytecode can be run on any computer, as long as they are running a JVM. This allows sharing code without sharing the full source code.

This is referred to as JIT compilation (Just-In-Time Compilation), because the full compilation is only done when the code is needed, as the code runs, but still stored as an object code file. The object code is then stored, meaning that it does not need to be recompiled on later uses. Because it is JIT, only parts of the code that are used in that specific compilation are used (subroutines, etc.), meaning that they will need compiling in the future if they are used.

6.2 Operating Systems

6.3 Programming Language Classification

6.3.1 Machine Code

Machine code is the binary 1s and 0s that enter a computer's CPU and control what it does. Machine code is rarely written by humans, since it is very difficult to read and write, however the first programs, written as holes on punch cards were written directly in machine code.

6.3.2 Assembly

Assembly was created as a more readable form of machine code. Assembly uses opcodes and operands to make instructions.

An opcode is a 3-letter mnemonic, short for a command, such as 'store' (`STR`). The opcode is then given operands, which are pieces of data that the command uses, for example, `STR` takes a register `Rn` and a memory reference, where it will store the contents of the `n`th register. Operands are either a memory address (written as a decimal number), a register (written with a capital R before a decimal number), or a value (written with a hash before a decimal number). To store the value in register 2 into memory location 300, assembly would use the line:

```
STR R2 300
```

The full instruction set for AQA assembly language can be found here. This is an example of a program in assembly:

```
// Compute the 10th Fibonacci number
LDR R0 #0   // R0 will be the counter to check how many
            // iterations have happened
LDR R3 #0   // R1 is the first number to add; it will be
            // shifted into R3
LDR R1 #1   // R2 is the second number to add

fibonacci:          // A label to start the loop
    CMP R0 #9       // Check whether more than 9 iterations
                    // have happened yet
    BGT exit        // If yes, exit
    LDR R2 R1       // Shift R1 into R2
    LDR R1 R3       // Shift R3 (previous sum) into R1
    ADD R3 R1 R2    // Adds the two previous numbers
    ADD R0 R0 #1    // Increments the counter
    B fibonacci     // Loops back to the fibonacci label

exit:               // Exit label for when the program
                    // finishes
```

```
STR R3 100      // Stores the result to memory location
                // 100
HALT            // Stop the program
```

Assembly has a one-to-one correspondance with machine code. This means that it can be converted, one instruction mapping to one machine code instruction. For example, the command `ADD R3 R1 R2` could be assembled into the machine code 0011 0011 0001 0010; the opcode (`ADD`) assembling to 0011, and the registers assembling to 0011 (`R3`), 0001 (`R1`), and 0010 (`R2`). This makes it very easy to convert into machine code, hence many early programs were written in it, but it makes it hard to read, even if it is easier than pure machine code. However, machine code gives the programmer complete control over the memory, down to each bit. This means that programs can be optimised perfectly for the CPU, and leave no wasted memory, whereas running code through higher-level languages can cause unnecessary machine code being added.

Each processor has its own assembly instruction set, which will be converted to machine code that that processor understands. This means that assembly can only work on one type of processor, whereas high-level languages will work for any processor that has a suitable interpreter/compiler for it.

Assembly code is often used in real-time systems. Real-time systems are systems that constantly monitor data from a set of sensors.

Device drivers are also generally written in assembly, because they must be fast to interpret inputs instantaneously.

Interrupt handlers are also written in assembly, because they run on a specific processor and are quite simple, low-level programs.

Overall, these qualities make assembly good for highly specific programs that run on one chip, but have to run very quickly. For example, an integrated system might use programs written in assembly, since they will waste little to no memory, will only have to run on that one chip, and have to run efficiently.

6.3.3 High-Level Languages

A high-level language is a language that looks more like English than machine code or assembly. The first high-level languages were languages such as Fortran, created by John Backus to look more like English. The program above to compute the 10th Fibonacci number looks like this in Fortran:

```
! Compute the 10th Fibonacci number
program fibonnaci
    integer :: n, i, a, b, sum  ! Declare all variables

    n = 10      ! Initisalise max iteration to 10
    a = 0       ! Set the first number to 0
    b = 1       ! Set the second number to 1

    do i = 2, n         ! Iterate from i = 2 to i = n
```

```
        sum = a + b     ! Add a to b
        a = b           ! Shift b to a
        b = sum         ! Shift sum to b
    end do

    print *, sum        ! Output the final sum
end program fibonnaci
```

This shows how much easier to read high-level languages are over low-level languages such as assembly. Variables mean that values can have meaningful names rather than just numbers, and structures such as iteration (`do i = 2, n`) allow programs to be much shorter by handling multiple assembly commands in one line.

High-level languages mean that programs are easier to code, decreasing production time and making the code more maintainable for future programmers.

6.3.4 Interpreted Languages

Some high-level languages are interpreted, meaning that they are read by an interpreter program line by line, and executed line by line as they are read. This means that programs have to be reread every time they are executed, which will slow them down. An example of an interpreted language is Python.

Interpreted languages require reinterpreting every time they are run, meaning that they can only be run if the person running it has an the interpreter and the source code, which means that the code can be easily copied by anyone with access to it. This makes it not ideal for large programs that are being sold.

6.3.5 Compiled Languages

Compiled languages are read in their entirety by a compiler, which compiles the source code (the code written by the developer) into a file of obeject code (machine code) called an executable. This can be directly run by a CPU, meaning that it is quick to run. However, any small edit to the code requires a full recompile, meaning that development can be slow for large projects that need a lot of recompiles. Some examples of compiled languages are VB.Net, C++, Java and Fortran.

Compiled code can be shared easily as an executable, meaning that the source code doesn't have to be shared as well as the program, making it ideal for a lot of application software.

6.4 Boolean Algebra

6.4.1 Truth Tables

A Boolean expression will take a set of inputs (usually written A, B, C,...), and return an output (usually written Q), all of which are Boolean values (true or false). This means that the set of possible values of the expression can be written as a table with columns A, B, C, etc. and Q, showing the outputs, like this:

A	**B**	**C**	$\cdots$	**Q**
0	0	0	$\cdots$	0
0	0	1	$\cdots$	1
0	1	0	$\cdots$	1
0	1	1	$\cdots$	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

When listing the inputs, they are listed as if they are multi-digit binary numbers from 0 to 2^n, so a 2-input truth table would list the inputs as 00, 01, 10, 11, in that order. If two truth tables are the same, then the Boolean expressions that made them are equivalent. For Boolean expressions ϕ and ψ, this is written as $\phi \iff \psi$, meaning that ϕ is true if and only if ψ is true.

6.4.2 Logic Gates

NOT Gates

A NOT gate, also known as an inverter, flips a true to a false and vice versa. NOT A is written as:

$$\text{NOT } A = \overline{A} \tag{6.1}$$

The truth table is:

A	**Q**
0	1
1	0

OR Gates

An OR gate outputs true if either output or both are true. A OR B is written as:

$$A \text{ OR } B = A + B \tag{6.2}$$

The truth table is:

A	**B**	**Q**
0	0	0
0	1	1
1	0	1
1	1	1

AND Gates

An AND gate outputs true if both inputs are true. A AND B is written as:

$$A \text{ AND } B = A \cdot B \tag{6.3}$$

The truth table is:

A	**B**	**Q**
0	0	0
0	1	0
1	0	0
1	1	1

XOR Gates

An XOR gate (exclusive OR gate) is true if one input is true, but not both. This can also be thought of as being true if and only if both inputs are not equal. A XOR B is written as:

$$A \text{ XOR } B = A \oplus B \tag{6.4}$$

NAND Gates

A NAND gate is an AND gate followed by a NOT gate, meaning that A NAND B can be written as:

$$A \text{ NAND } B = \text{NOT } (A \text{ AND } B) = \overline{A \cdot B} \tag{6.5}$$

The truth table is:

A	**B**	**Q**
0	0	1
0	1	1
1	0	1
1	1	0

NAND is referred to as the universal gate, because it can be used to make any other logic gate. These can be built as:

$$\text{NOT } A \text{ NAND } A \tag{6.6}$$

NOR gates can also be used to make any other logic gate.

NAND gates are used to build other gates, because this lowers cost of manufacturing, since only one gate has to be made. Using as few gates as possible speeds up processing.

NOR Gates

A NOR gate is an OR gate followed by a NOT gate, meaning that A NOR B can be written as:

$$A \text{ NOR } B = \text{NOT } (A \text{ OR } B) = \overline{A + B} \tag{6.7}$$

The truth table is:

A	**B**	**Q**
0	0	1
0	1	0
1	0	0
1	1	0

Logic Gate Diagrams

Each logic gate has a symbol that can be used to draw diagrams that represent Boolean expressions. The symbols are:

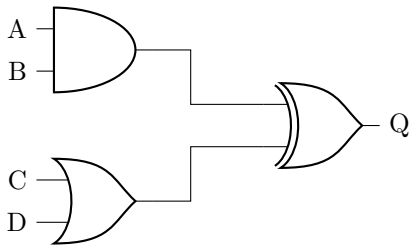


Figure 6.1: A logic diagram

6.4.3 De Morgan's Laws

Theorem 6.4.1 (De Morgan's First Law)**.**

$$\overline{A} \cdot \overline{B} = \overline{A + B} \tag{6.8}$$

Proof. De Morgan's first law can be proved with a truth table:

A	**B**	**A + B**	$\overline{A + B}$	$\overline{A} \cdot \overline{B}$	$\overline{B}$	$\overline{A}$
0	0	0	1	1	1	1
0	1	1	0	0	1	0
1	0	1	0	0	0	1
1	1	1	0	0	0	0

□

De Morgan's second law:

Theorem 6.4.2 (De Morgan's Second Law)**.**

$$\overline{A \cdot B} = \overline{A} + \overline{B} \tag{6.9}$$

Proof. De Morgan's second law can be proved with a truth table. On the left, there are the values of A and B, which are then used to find $A \cdot B$ and then $\overline{A \cdot B}$. On the right, there are the values of $\overline{A}$ and $\overline{B}$, which are then used to find $\overline{A} + \overline{B}$.

A	**B**	$A \cdot B$	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$	$\overline{B}$	$\overline{A}$
0	0	0	1	1	1	1
0	1	0	1	1	1	0
1	0	0	1	1	0	1
1	1	1	0	0	0	0

Since the columns for $\overline{A \cdot B}$ and $\overline{A} + \overline{B}$, the two expressions must be equal. □

De Morgan's laws can be applied to an entire expression, or to smaller part of it.

6.4.4 Other Laws of Boolean Algebra

For any boolean variables $A, B, C, D \in \mathbb{B} = \{0, 1\}$, the following identities apply:

$$A \cdot 0 = 0 \tag{6.10}$$
$$A \cdot 1 = A \tag{6.11}$$
$$A \cdot A = A \tag{6.12}$$
$$A \cdot \overline{A} = 0 \tag{6.13}$$

$$A + 0 = A \tag{6.14}$$
$$A + 1 = 1 \tag{6.15}$$
$$A + A = A \tag{6.16}$$
$$A + \overline{A} = 1 \tag{6.17}$$

$$A \cdot B = B \cdot A \tag{6.18}$$
$$A + B = B + A \tag{6.19}$$

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C \tag{6.20}$$
$$A + (B + C) = (A + B) + C \tag{6.21}$$

$$A \cdot (B + C) = A \cdot B + A \cdot C \tag{6.22}$$
$$(A + B) \cdot (C + D) = A \cdot C + A \cdot D + B \cdot C + B \cdot D \tag{6.23}$$

$$A \cdot (A + B) = A \tag{6.24}$$
$$A + (A \cdot B) = A \tag{6.25}$$

Expressions using AND can be written with no symbols, e.g.:

$$A \cdot (B) = A(B) \tag{6.26}$$

This is an example of simplifying a Boolean expression:

$(A + B) \cdot C \cdot \overline{C} + (A + \overline{A}) \cdot B$	$C \cdot \overline{C} = 0$
$(A + B) \cdot 0 + (A + \overline{A}) \cdot B$	$(A + B) \cdot 0 = 0$
$0 + (A + \overline{A}) \cdot B$	$A + \overline{A} = 1$
$0 + 1 \cdot B$	$1 + B = B$
$0 + B$	$0 + B = B$
B	

The operations are treated like + and · in elementary algebra, where + is addition, and · is multiplication. This means that · comes before + in the order of operations. NOT groups a subexpression into a piece, which can be treated as a bracket, meaning that the contents of a NOT group is evaluated before anything else, along with brackets. XOR comes after NOT/brackets, and before AND, but is unlikely to appear in questions.

The Boolean algebra thus has commutativity for · and +, associativity for · and +. It also has an identity for · and + ($I_\cdot = 1$; $I_+ = 0$), just like the identities for · and + in elementary algebra.

6.4.5 Order of Operations

B	Brackets/NOT
N/A	XOR
I	N/A
D/M	AND
A/S	OR

Boolean algebra uses BIDMAS just like elementary algebra; AND is multiplication (hence it uses the · symbol); OR is addition (hence it uses the + symbol); and NOT groups terms into subexpressions like brackets.

Chapter 7

Organisation and Architecture

Chapter 8

Consequences of Use

Chapter 9

Communication and Networking

Chapter 10

Databases

Chapter 11

Big Data

Chapter 12

Functional Programming

Chapter 13

Problem Solving